

Phase 1 Execution Plan: Data Preprocessing and System Initialization

Current Status: Raw aquaculture PDF manuals are currently stored in the shared Google Drive repository.

Objective: The immediate objective is to systematically clean the collected documents, execute the semantic chunking algorithms, and construct the offline vector database.

Task Distribution and Workflow Sequence

1. Data Preprocessing and Execution (Assigned to M4: Praisyy)

Role: Data Owner

The data pipeline requires pristine text inputs. Extraneous formatting and non-machine-readable elements will cause severe downstream extraction errors.

- **Initial Download and Verification:** Raw PDFs must be downloaded from the shared repository to a local machine.
- **Text Extractability Validation:** Each document must be opened and tested for text highlighting. Pages that consist of scanned images and cannot be highlighted must be extracted and deleted using PDF modification tools.
- **Extraneous Content Removal:** Sections such as the Table of Contents, Title Pages, Indexes, and Reference lists must be deleted. Only paragraphs containing factual aquaculture data (such as parameters indicating dissolved oxygen levels above 4 ppm or water pH limits between 6.5 and 9) are to be retained.
- **File Standardization:** Files must be renamed using a standardized, lowercase format without spaces (e.g., `icar_shrimp_manual.pdf`).
- **Execution Handoff:** Once all PDFs are cleaned and compiled into a single directory, the team must be notified. Execution will pause until the ingestion script is provided by the Core ML Architect.

2. Core ML Pipeline Implementation (Assigned to M1: Khushi)

Role: Machine Learning Architect

Simultaneous to the data cleaning process, the Python logic required for document processing must be developed.

- **Ingestion Script Development (`ingest.py`):** Code must be written to open the cleaned PDFs, extract the text, and segment it into 150-word blocks. A 30-word overlap is required to ensure contextual continuity. Rule-based metadata tags (e.g., "Ammonia" or "Shrimp") will be appended to these blocks.
- **Embedding Script Development (`embed.py`):** Code must be developed to transform the text blocks into dense vectors using the BGE-M3 model. These vectors will then be indexed into a local ChromaDB instance.
- **Deployment:** Upon successful testing on a sample document, the code will be pushed to the project GitHub repository. The Data Owner will be instructed to execute the scripts on the cleaned PDF directory.

3. Integration and Retrieval Testing (Assigned to M2: Meghana)

Role: Quality Assurance (QA) Tester

Testing the database is dependent on the completion of the ChromaDB index by the Data Owner.

Preparations for testing must begin immediately.

- **Test Case Design:** A spreadsheet must be created containing 10 to 15 highly specific English queries representative of actual agricultural inquiries (e.g., questions regarding the optimal dissolved oxygen level for *Litopenaeus vannamei* or treatments for White Spot disease).
- **Database Integration:** Upon receiving the compiled `chroma_db` directory from the Data Owner, a testing script must be authored.
- **System Validation:** The testing script will execute the prepared queries against the database. The accuracy of the retrieved paragraphs must be evaluated and logged as a pass or fail metric.

4. Documentation and Architecture Visuals (Assigned to M3: Jyothish)

Role: Researcher and Project Manager

Documentation tasks can commence immediately using the raw PDFs currently available in the shared repository.

- **Citation Management:** All downloaded PDFs must be imported into a citation manager (e.g., Zotero). Metadata such as titles, authors (e.g., ICAR-CIBA), and publication dates must be accurately entered to facilitate automated IEEE citation generation.
 - **Project Documentation:** The shared project logbook must be updated with the current week's progress. It must be noted that manual PDF preprocessing is underway to eliminate non-machine-readable pages, while the semantic chunking logic is concurrently being designed.
 - **Architectural Diagramming:** A technical flowchart detailing the Phase 1 process must be created using a diagramming tool. The diagram must accurately depict the following sequential flow: Raw PDF Manuals -> Manual Preprocessing -> PyMuPDF Semantic Chunking -> BGE-M3 Multilingual Embedding -> ChromaDB Vector Store.
-

Repository Architecture and Version Control Guidelines

Overview: To maintain an organized, conflict-free codebase for the aquaculture RAG system, a centralized GitHub repository has been established. All code, processed text data, and documentation must adhere to the following directory structure and version control rules.

1. Directory Structure Specifications

The repository is divided into isolated directories to separate executable code, data pipelines, and academic documentation.

- **src/:** Dedicated exclusively to Python source code. All executable scripts (`ingest.py`, `embed.py`, `test_retrieval.py`) must be saved here.
- **data/raw_pdfs/:** Designated for the cleaned, manual PDF files. This directory is strictly local.
- **data/processed/:** Designated for the output of the ingestion scripts (e.g., `chunks.json`). This lightweight text data will be tracked by Git to ensure all team members have access to the exact same baseline text corpus.
- **data/chroma_db/:** Designated for the compiled vector database. This directory is strictly local.
- **docs/:** Designated for project management artifacts, including Zotero bibliographies, architectural flowcharts, and manuscript drafts.

2. Version Control Constraints (.gitignore Policy)

Machine learning projects generate massive temporary files that can corrupt Git repositories. A `.gitignore` file has been configured to enforce the following data rules:

- **No Binary Uploads:** Raw PDF files must never be committed to the repository. They must remain in the shared Google Drive or stored locally on individual machines.
- **No Database Uploads:** The `chroma_db` folder contains heavy SQLite files and index graphs. This folder must not be committed. It must be generated locally by running `embed.py`, or shared via compressed ZIP files if manual transfer is required.
- **Allowed Data:** Only the cleaned `.json` files located within the `data/processed/` folder are permitted to be pushed to the remote repository.

3. Execution Workflow

All team members must execute a **git pull** prior to beginning local work to ensure synchronization with the latest script modifications. Code modifications must be committed with clear, descriptive messages detailing the exact logic changed or bugs resolved.

Repository Structure

aquassist/

```
|— .gitignore      # Instructs Git to ignore PDFs and databases
|— README.md       # Project setup instructions
|— requirements.txt # List of Python dependencies
|— data/
|   |— raw_pdfs/    # Local use only (Ignored by Git)
|   |— processed/   # Contains chunks.json (Tracked by Git)
|   |— chroma_db/   # Local use only (Ignored by Git)
|— docs/           # IEEE drafts, literature, diagrams
|— src/            # Core Python scripts
|   |— ingest.py
|   |— embed.py
|   |— test_retrieval.py
```